

SPECYFIKACJA WYMAGAŃ I DOKUMENTACJA TECHNICZNA DLA SYSTEMU GYMAPP

Aplikacja mobilna do planowania, wykonywania i analizowania treningów siłowych

Wersja dokumentu: 1.0

Data opracowania: 17.06.2026

Repozytorium: github.com/Vkosepp/GymApp

Zakres: wymagania, architektura, model danych, procesy, interfejsy, testy i rekomendacje

Dokument przygotowany w estetycznym układzie technicznym, z czytelnymi diagramami i tabelami dzielonymi na krótkie sekcje.

Historia zmian dokumentu

Osoba	Data	Komentarz	Wersja
Autor opracowania	17.06.2026	Pierwsza pełna wersja dokumentacji technicznej i specyfikacji wymagań dla GymApp.	1.0

Spis treści

- 1. Wprowadzenie
 - 1.1. Cel dokumentu
 - 1.2. Przyjęte zasady w dokumencie
 - 1.3. Zakres produktu
 - 1.4. Literatura i źródła
- 2. Opis ogólny
 - 2.1. Perspektywa produktu
 - 2.2. Funkcje produktu
 - 2.3. Ograniczenia
 - 2.4. Dokumentacja użytkownika
 - 2.5. Założenia i zależności
- 3. Model procesów biznesowych
 - 3.1. Aktorzy
 - 3.2. Obiekty biznesowe
 - 3.3. Procesy biznesowe
 - 3.4. Reguły biznesowe
- 4. Wymagania funkcjonalne
- 5. Charakterystyka interfejsów
- 6. Wymagania pozafunkcjonalne
- 7. Architektura techniczna
- 8. Model danych
- 9. Testowanie i jakość
- 10. Wdrożenie i uruchomienie
- 11. Lista spraw otwartych
- Dodatek A. Słownik pojęć
- Dodatek B. Słownik danych
- Dodatek C. Modele analizy

1. Wprowadzenie

1.1. Cel dokumentu

Celem dokumentu jest uporządkowany, techniczny opis projektu GymApp: aplikacji mobilnej przeznaczonej do planowania treningów, prowadzenia aktywnej sesji ćwiczeń, zapisywania historii oraz obserwowania postępów użytkownika. Dokument łączy specyfikację wymagań z dokumentacją techniczną aktualnej implementacji dostępnej w repozytorium.

Dokument jest przeznaczony dla prowadzącego, osób oceniających projekt, przyszłych programistów rozwijających aplikację oraz dla użytkownika technicznego, który chce zrozumieć, z jakich modułów składa się system i jak przepływają w nim dane.

Zakres analizy

Opis obejmuje widoczną strukturę projektu, kod Kotlin, warstwę Android/Compose, lokalną bazę Room, repozytorium danych, ViewModele, ekrany aplikacji, diagramy, wymagania funkcjonalne i pozafunkcjonalne oraz wskazane ograniczenia bieżącej wersji.

1.2. Przyjęte zasady w dokumencie

Wymagania zapisano jako krótkie, jednoznaczne pozycje z identyfikatorem, priorytetem i sposobem weryfikacji. Priorytety oznaczają: W - wymagane do działania podstawowego, P - pożądane, R - rozwojowe. Nazwy klas, plików i tabel bazy danych zapisano czcionką techniczną lub w cudzysłowie, aby nie mieszać ich z nazwami biznesowymi.

- Nazwy ekranów odpowiadają trasom nawigacji z klasy Screen, np. home, plans, stats, profile, settings.
- Nazwy tabel odpowiadają adnotacjom @Entity użytym w warstwie Room, np. workout_plans, plan_exercises, performed_sets.
- Diagramy są celowo podzielone na kilka osobnych rysunków, aby żaden z nich nie był przeładowany i aby elementy nie zachodziły na siebie.

1.3. Zakres produktu

GymApp jest aplikacją mobilną działającą lokalnie na urządzeniu Android. Produkt umożliwia zbudowanie własnej bazy planów treningowych na podstawie predefiniowanego katalogu ćwiczeń, przypisywanie planów do dni w kalendarzu, uruchamianie treningu, rejestrowanie wykonanych serii oraz przeglądanie statystyk i zdjęć progresu.

System nie jest aplikacją sieciową i nie korzysta z backendu. Dane treningowe są przechowywane lokalnie w bazie SQLite zarządzanej przez Room, a zdjęcia profilu i progresu są kopiowane do prywatnej pamięci aplikacji. Oznacza to, że projekt jest prosty w uruchomieniu, ale wymaga świadomego podejścia do migracji danych i kopii zapasowych w przyszłych wersjach.

1.4. Literatura i źródła

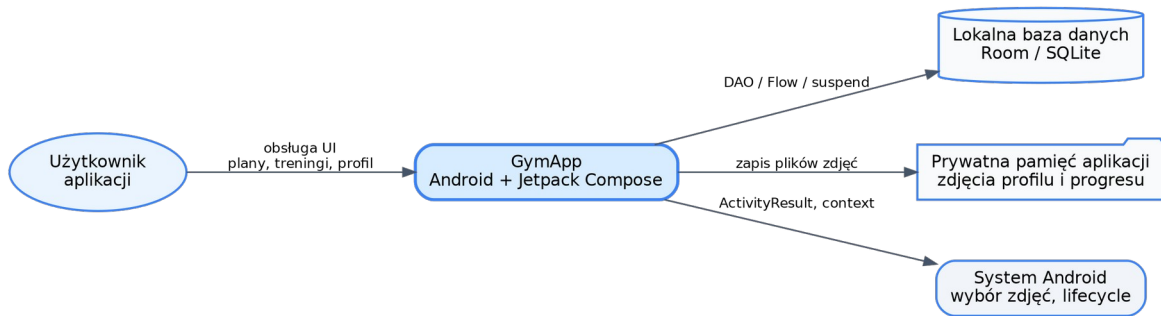
ID	Źródło	Wykorzystanie
ŻR-01	Repozytorium GitHub Vkosepp/GymApp	Kod źródłowy projektu: struktura modułów, pliki Kotlin, konfiguracja Gradle.
ŻR-02	Specyfikacja wymagań - szablon i przykład ADE	Układ dokumentu: wprowadzenie, opis ogólny, procesy, wymagania, interfejsy, wymagania pozafunkcjonalne, dodatki.
ŻR-03	Instrukcja specyfikacji wymagań dla systemów informatycznych	Zasady opisu wymagań: kompletność, jednoznaczność, weryfikowalność, priorytetyzacja i śledzenie powiązań.

ŻR-04	Dokumentacja Android / Jetpack Compose / Room	Kontekst technologiczny dla Activity, Compose, ViewModel, StateFlow, Room DAO i SQLite.
-------	---	---

2. Opis ogólny

2.1. Perspektywa produktu

GymApp funkcjonuje jako samodzielna aplikacja Android. Z punktu widzenia użytkownika jest to cyfrowy dziennik treningowy z kalendarzem, listą planów, aktywnym trybem treningu, profilem, galerią progresu i statystykami. Z punktu widzenia architektury jest to aplikacja jednowarstwowo wdrażana na telefonie, ale logicznie podzielona na warstwę UI, ViewModel, repozytorium i lokalną bazę danych.



Rysunek 1. Diagram kontekstu systemu GymApp.

2.2. Funkcje produktu

- prowadzenie profilu użytkownika z imieniem/nickiem, wiekiem, wzrostem i awatarem;
- prezentacja katalogu ćwiczeń z podziałem na partie mięśniowe oraz wyszukiwaniem po nazwie;
- tworzenie i edycja planów treningowych składających się z ćwiczeń i planowanych serii;
- obsługa trybu zaawansowanego planu z tempem ruchu: faza ekscentryczna, izometryczna i koncentryczna;
- kalendarz miesięczny z przypisywaniem planów do konkretnych dat;
- uruchomienie treningu dla planu przypisanego do dzisiejszego dnia;
- prowadzenie aktywnej sesji: stoper całego treningu, minutnik przerwy, wpisywanie kg i liczby powtórzeń;
- zapis historii wykonanych serii i sesji treningowych;
- statystyki: rozkład ćwiczeń według grup mięśniowych oraz wykres postępu dla wybranego ćwiczenia;
- galeria zdjęć progresu z opcjonalną wagą oraz reset danych użytkownika.

2.3. Ograniczenia

ID	Ograniczenie	Typ
OGR-01	Aplikacja działa lokalnie; brak kont użytkowników, logowania i synchronizacji między urządzeniami.	Techniczne
OGR-02	Baza danych używa fallbackToDestructiveMigration, więc nieobsłużona migracja może usunąć lokalne dane.	Dane
OGR-03	Ustawienia powiadomień, wibracji i niewygaszania ekranu są w bieżącej wersji stanami aplikacji, a nie pełną integracją systemową.	Funkcjonalne
OGR-04	Historia treningu jest zapisywana lokalnie; brak eksportu/importu danych.	Funkcjonalne

OGR-05	Statystyki postępu dla dat w aktualnym kodzie mają uproszczenie demonstracyjne w zakresie etykiet dat.	Analityczne
OGR-06	Projekt nie zawiera widocznego modułu backendowego, panelu administracyjnego ani zdalnego API.	Architektura

2.4. Dokumentacja użytkownika

Nazwa	Opis zawartości	Format	Język
Instrukcja użytkownika GymApp	Opis ekranów: kalendarz, plany, wybór ćwiczeń, edytor, trening, statystyki, profil i ustawienia.	PDF / DOCX	polski
Instrukcja techniczna uruchomienia	Wymagania środowiska, import projektu do Android Studio, uruchomienie emulatora lub telefonu.	PDF / DOCX	polski
Dokumentacja techniczna	Architektura, model danych, wymagania, interfejsy, testy i rekomendacje rozwojowe.	DOCX / PDF	polski

2.5. Założenia i zależności

- Użytkownik korzysta z urządzenia Android spełniającego minimalną wersję SDK 24.
- Projekt jest budowany przez Gradle/Kotlin DSL jako moduł :app.
- Interfejs użytkownika jest zbudowany w Jetpack Compose i Material 3.
- Dane utrwalane są przez Room w lokalnej bazie SQLite o nazwie gym_database.
- Zdjęcia użytkownika są kopiowane do prywatnego katalogu aplikacji, a w bazie przechowywana jest ścieżka pliku.

3. Model procesów biznesowych

3.1. Aktorzy i charakterystyka użytkowników

ID	Aktor	Charakterystyka
A-01	Użytkownik aplikacji	Osoba ćwicząca, która tworzy plany treningowe, przypisuje je do kalendarza, wykonuje treningi i śledzi progres.
A-02	System Android	Dostarcza środowisko uruchomieniowe, cykl życia Activity, pamięć prywatną aplikacji oraz mechanizm wyboru zdjęć.
A-03	Baza Room/SQLite	Lokalny komponent przechowujący profil, ćwiczenia, plany, serie, sesje, zdjęcia i zaplanowane treningi.
A-04	Programista / tester	Osoba rozwijająca projekt, uruchamiająca testy, diagnozująca błędy i wykonująca migracje danych.

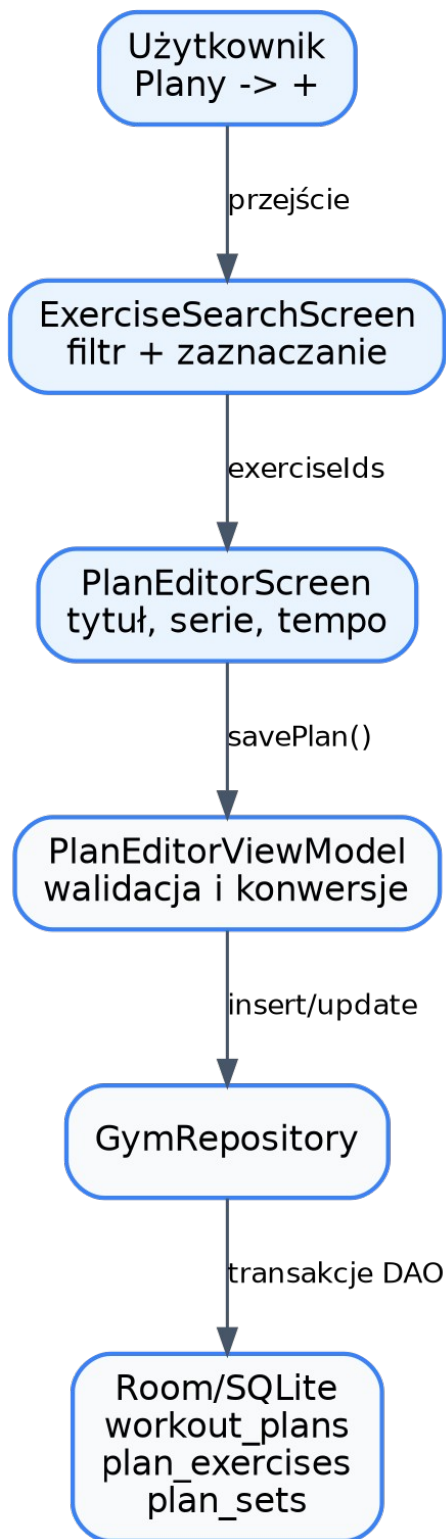
3.2. Obiekty biznesowe

Obiekt	Opis
Ćwiczenie	Pozycja w słowniku ćwiczeń, określona nazwą i grupą mięśniową.
Plan treningowy	Szablon treningu nazwany przez użytkownika, np. „Push”, „Nogi”, „FBW”.
Ćwiczenie w planie	Powiązanie planu z ćwiczeniem oraz trybem podstawowym lub zaawansowanym.
Seria planowana	Docelowa seria w planie: numer serii, liczba powtórzeń i ciężar.
Trening zaplanowany	Przypisanie planu do konkretnej daty kalendarzowej.

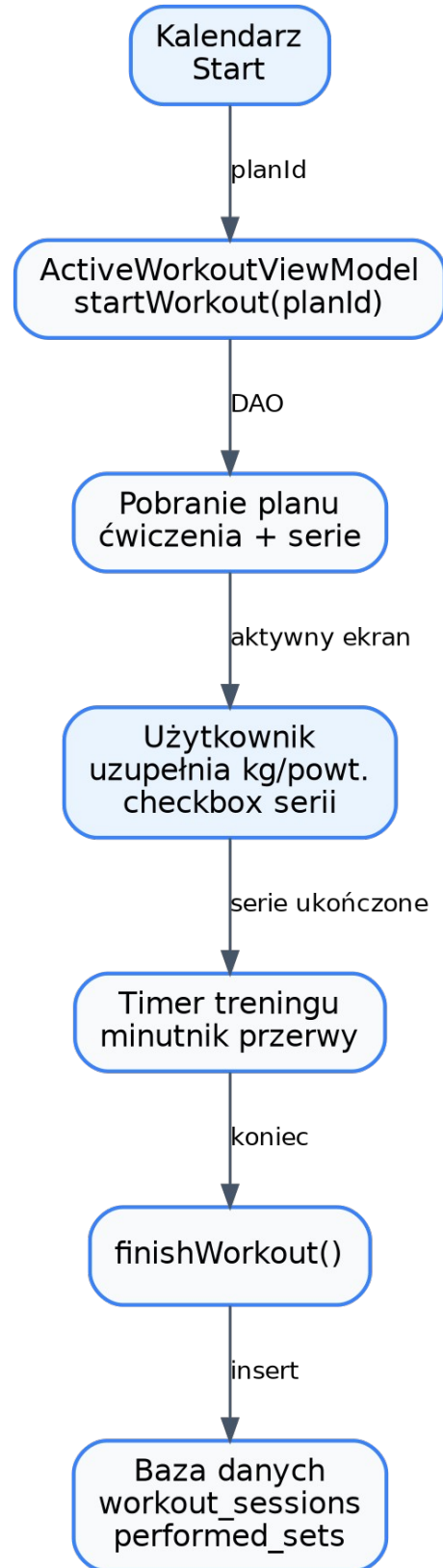
Sesja treningowa	Jedno wykonanie planu, zapisane z datą i czasem trwania.
Seria wykonana	Rzeczywiście wykonana seria: ćwiczenie, numer serii, powtórzenia, ciężar.
Zdjęcie progresu	Zdjęcie zapisane w prywatnej pamięci aplikacji z datą i opcjonalną wagą.
Profil użytkownika	Dane podstawowe użytkownika oraz ścieżka awatara.

3.3. Procesy biznesowe

ID	Proces	Opis
PB-01	Utworzenie planu treningowego	Użytkownik wybiera ćwiczenia, przechodzi do edytora, podaje tytuł planu, parametry serii i zapisuje plan.
PB-02	Edycja planu treningowego	Użytkownik otwiera istniejący plan, modyfikuje tytuł oraz parametry ćwiczeń, a system zastępuje poprzednie ćwiczenia i serie nowymi.
PB-03	Zaplanowanie treningu	Użytkownik wybiera datę w kalendarzu, wybiera plan i przypisuje go do dnia.
PB-04	Wykonanie treningu	Użytkownik uruchamia plan z kalendarza, uzupełnia serie, zaznacza wykonanie, korzysta z przerwy i kończy sesję.
PB-05	Analiza postępów	Użytkownik wybiera ćwiczenie i filtr czasu, a system prezentuje wykres maksymalnego ciężaru oraz szacowanego 1RM.
PB-06	Dokumentowanie progresu zdjęciem	Użytkownik wybiera zdjęcie, opcjonalnie podaje wagę, a system zapisuje kopię pliku i metadane.
PB-07	Reset danych użytkownika	Użytkownik potwierdza operację w ustawieniach, a system usuwa plany, historię, kalendarz, zdjęcia i profil.



Rysunek 2. Proces tworzenia planu.



Rysunek 3. Proces wykonania treningu.

3.4. Reguły biznesowe

ID	Definicja reguły	Typ	Zmienność
RB-01	Plan treningowy musi mieć tytuł; jeżeli użytkownik go nie zmieni, używana jest wartość domyślna „Nowy Plan #1”.	Ograniczenie	Statyczna
RB-02	Plan może zawierać wiele ćwiczeń, a każde ćwiczenie w planie może mieć wiele serii planowanych.	Fakt	Statyczna
RB-03	W trybie podstawowym użytkownik definiuje liczbę serii, powtórzenia i ciężar; system generuje odpowiednie rekordy serii.	Obliczenie	Statyczna
RB-04	W trybie zaawansowanym ćwiczenie w planie przechowuje trzy wartości tempa: ekscentryczne, izometryczne i koncentryczne.	Fakt	Statyczna
RB-05	Trening można wystartować z kalendarza tylko dla aktualnego dnia, zgodnie z warunkiem interfejsu.	Ograniczenie	Statyczna
RB-06	Zaznaczenie wykonanej serii uruchamia domyślny minutnik przerwy 90 sekund.	Wyzwalacz	Statyczna
RB-07	Tylko serie oznaczone jako wykonane są zapisywane jako performed_sets po zakończeniu treningu.	Ograniczenie	Statyczna
RB-08	Zdjęcie progresu może mieć wagę, ale nie musi; wtedy pole weight pozostaje puste.	Fakt	Statyczna
RB-09	Reset danych użytkownika nie usuwa słownika ćwiczeń, ponieważ stanowi on bazę startową aplikacji.	Ograniczenie	Statyczna

4. Wymagania funkcjonalne

Wymagania funkcjonalne zostały podzielone na moduły odpowiadające realnym ekranom i klasom aplikacji. Każdy moduł opisuje funkcję widoczną dla użytkownika oraz sposób, w jaki obecna implementacja ją realizuje.

M01. Profil użytkownika

Pole	Opis
Opis modułu	Moduł pozwala przechowywać dane profilu: imię/nick, wiek, wzrost i awatar. Użytkownik może także prowadzić galerię progresu ze zdjęciami i opcjonalną wagą.
Priorytet	W
Dane wejściowe	Dane tekstowe profilu, URI zdjęcia, opcjonalna waga.
Dane wyjściowe	Rekord UserEntity, rekord ProgressPhotoEntity, plik w prywatnym katalogu aplikacji.

ID	Przypadek użycia	Aktor	Scenariusz podstawowy	Wyjątki / uwagi
UC-01	Edycja profilu	Użytkownik	Otwiera profil, wybiera edycję, wpisuje dane i zapisuje.	Błędne liczby są zastępowane wartością 0.
UC-02	Zmiana awatara	Użytkownik	Wybiera obraz z urządzenia; system kopiuje go do prywatnej pamięci.	Błąd odczytu pliku powoduje brak zmiany awatara.
UC-03	Dodanie zdjęcia progresu	Użytkownik	Wybiera zdjęcie, opcjonalnie podaje wagę, system zapisuje metadane.	Waga może zostać pominięta.

M02. Słownik i wyszukiwanie ćwiczeń

Pole	Opis
Opis modułu	Moduł prezentuje predefiniowaną bazę ćwiczeń, pozwala filtrować po grupie mięśniowej i wyszukiwać po fragmencie nazwy.
Priorytet	W
Dane wejściowe	Tekst wyszukiwania, wybrana grupa, identyfikatory ćwiczeń.
Dane wyjściowe	Lista przefiltrowanych ExerciseEntity i lista wybranych ID.

ID	Przypadek użycia	Aktor	Scenariusz podstawowy	Wyjątki / uwagi
UC-04	Wyszukanie ćwiczenia	Użytkownik	Wpisuje tekst w pole wyszukiwania; lista zawęży wyniki.	Pusta fraza pokazuje wszystkie ćwiczenia z wybranej grupy.
UC-05	Filtrowanie po partii mięśniowej	Użytkownik	Wybiera zakładkę All, Legs, Chest, Back, Shoulders, Arms lub Core.	Ćwiczenia spoza tych zakładek są mniej widoczne w UI.
UC-06	Wybór ćwiczeń do planu	Użytkownik	Zaznacza checkboxy i przechodzi dalej.	Przycisk Dalej jest nieaktywny bez wyboru ćwiczeń.

M03. Zarządzanie planami treningowymi

Pole	Opis
Opis modułu	Moduł odpowiada za tworzenie, edycję, usuwanie i wyświetlanie planów treningowych. Plan składa się z nazwy, listy ćwiczeń i serii.
Priorytet	W
Dane wejściowe	Tytuł planu, wybrane ćwiczenia, serie, powtórzenia, ciężar, tempo.
Dane wyjściowe	WorkoutPlanEntity, PlanExerciseEntity i PlanSetEntity.

ID	Przypadek użycia	Aktor	Scenariusz podstawowy	Wyjątki / uwagi
UC-07	Utworzenie planu	Użytkownik	Wybiera ćwiczenia, uzupełnia parametry i zapisuje plan.	Niepoprawne wartości są zastępowane domyślnymi.
UC-08	Edycja planu	Użytkownik	Otwiera kartę planu, modyfikuje pola, zapisuje.	Stare ćwiczenia planu są usuwane i wstawiane ponownie.
UC-09	Usunięcie planu	Użytkownik	Potwierdza usunięcie planu na liście.	Kaskady usuwają powiązane ćwiczenia planu i serie planowane.

M04. Kalendarz i planowanie treningów

Pole	Opis
Opis modułu	Moduł kalendarza umożliwia przechodzenie między miesiącami, wybór daty, oznaczenie dni z treningami i przypisanie planu do dnia.
Priorytet	W
Dane wejściowe	Wybrana data i ID planu.
Dane wyjściowe	ScheduledWorkoutEntity z dateString w formacie ISO YYYY-MM-DD.

ID	Przypadek użycia	Aktor	Scenariusz podstawowy	Wyjątki / uwagi
UC-10	Wybór daty w kalendarzu	Użytkownik	Użytkownik klika dzień; system ustawia selectedDate.	Zmiana miesiąca aktualizuje currentMonth.
UC-11	Przypisanie planu do daty	Użytkownik	Kliknięcie plusa otwiera listę planów, kliknięcie planu zapisuje schedule.	Jeżeli nie ma planów, system pokazuje komunikat.
UC-12	Usunięcie zaplanowanego treningu	Użytkownik	Kliknięcie ikony kosza usuwa rekord scheduled_workouts.	Brak dodatkowego potwierdzenia w aktualnym scenariuszu karty.

M05. Aktywny trening

Pole	Opis
Opis modułu	Moduł prowadzi użytkownika przez wykonanie planu. Ładuje ćwiczenia i serie z bazy, uruchamia stoper, pozwala zmienić kg i powtórzenia oraz zapisuje tylko wykonane serie.
Priorytet	W
Dane wejściowe	ID planu, wykonane serie, rzeczywiste kg i powtórzenia.
Dane wyjściowe	Sesja treningowa, wykonane serie, czas trwania.

ID	Przypadek użycia	Aktor	Scenariusz podstawowy	Wyjątki / uwagi
UC-13	Start treningu	Użytkownik	Uruchamia trening z kalendarza; system ładuje plan i startuje stoper.	Brak planu lub pusty plan skutkuje pustą listą ćwiczeń.
UC-14	Oznaczenie serii	Użytkownik	Użytkownik zaznacza serię jako wykonaną; system uruchamia minutnik przerwy.	Odnaczenie przerywa minutnik.
UC-15	Zakończenie treningu	Użytkownik	System tworzy WorkoutSessionEntity i PerformedSetEntity dla wykonanych serii.	Pola tekstowe bez liczby są zapisane jako 0.

M06. Statystyki i analiza postępu

Pole	Opis
Opis modułu	Moduł pokazuje rozkład pracy według grup mięśniowych oraz dane postępu dla wybranego ćwiczenia: maksymalny ciężar i szacowany 1RM według wzoru Brzyckiego.
Priorytet	P
Dane wejściowe	Historia performed_sets i lista ćwiczeń.
Dane wyjściowe	Dane do wykresu radarowego i liniowego.

ID	Przypadek użycia	Aktor	Scenariusz podstawowy	Wyjątki / uwagi
UC-16	Wybór ćwiczenia do statystyk	Użytkownik	Wybiera ćwiczenie z listy rozwijanej.	Brak wyboru oznacza brak danych wykresu liniowego.
UC-17	Zmiana zakresu czasu	Użytkownik	Wybiera filtr 14 lub 28 dni.	Obsługiwane są tylko zdefiniowane filtry.
UC-18	Odczyt rozkładu mięśniowego	Użytkownik	System zlicza wykonane serie według mapowanych grup mięśniowych.	Nieznane grupy są mapowane do kategorii stretching/inne.

M07. Ustawienia

Pole	Opis
Opis modułu	Moduł przechowuje stan motywu i przełączników aplikacji oraz umożliwia reset danych użytkownika.
Priorytet	P
Dane wejściowe	Stany przełączników, potwierdzenie resetu.
Dane wyjściowe	Zmieniony ThemeState lub wyczyszczone tabele użytkownika.

ID	Przypadek użycia	Aktor	Scenariusz podstawowy	Wyjątki / uwagi
UC-19	Zmiana motywu	Użytkownik	Przełącza ciemny motyw; globalny ThemeState aktualizuje UI.	Stan może nie być trwały po restarcie, jeśli nie zostanie zapisany.
UC-20	Przełączenie preferencji	Użytkownik	Zmienia powiadomienia, niewygaszanie ekranu, wibracje.	Wymaga dalszej integracji z usługami Android.
UC-21	Reset danych	Użytkownik	Potwierdza dialog; system czyści dane użytkownika.	Operacja jest nieodwracalna w bieżącej wersji.

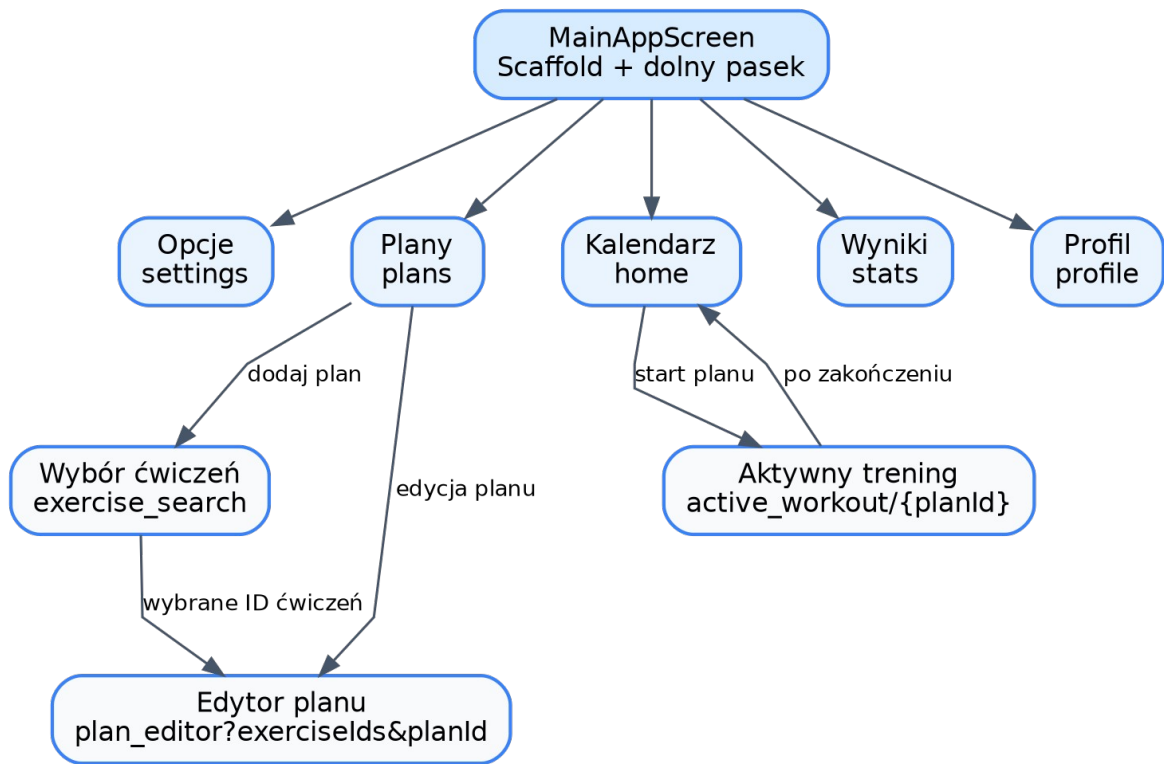
4.8. Zbiorcza lista wymagań funkcjonalnych

ID	Wymaganie	Priorytet	Weryfikacja
WF-01	System musi uruchamiać ekran główny przez MainActivity i MainAppScreen.	W	Uruchomienie aplikacji na emulatorze.
WF-02	System musi wyświetlać dolny pasek nawigacyjny dla ekranów: Opcje, Plany, Kalendarz, Wyniki, Profil.	W	Test UI na każdej trasie głównej.
WF-03	System musi przechowywać ćwiczenia w lokalnej bazie Room.	W	Inspekcja bazy i lista ćwiczeń po pierwszym uruchomieniu.
WF-04	System musi umożliwiać tworzenie planu z wielu ćwiczeń.	W	Scenariusz UC-07.
WF-05	System musi umożliwiać edycję istniejącego planu.	W	Scenariusz UC-08.
WF-06	System musi umożliwiać usunięcie planu.	W	Scenariusz UC-09.
WF-07	System musi umożliwiać przypisanie planu do daty.	W	Scenariusz UC-11.
WF-08	System musi umożliwiać start treningu z planu.	W	Scenariusz UC-13.
WF-09	System musi zapisywać tylko serie oznaczone jako wykonane.	W	Porównanie checkboxów z tabelą performed_sets.
WF-10	System powinien prezentować statystyki progresu dla wybranego ćwiczenia.	P	Wybór ćwiczenia i sprawdzenie wykresów.
WF-11	System powinien umożliwiać zapis zdjęć progresu.	P	Dodanie zdjęcia i kontrola wpisu progress_photos.
WF-12	System powinien umożliwiać reset danych użytkownika.	P	Potwierdzenie dialogu i weryfikacja pustych list.

5. Charakterystyka interfejsów

5.1. Interfejs użytkownika

Interfejs użytkownika jest zbudowany w Jetpack Compose. Główny ekran MainAppScreen tworzy Scaffold z dolnym paskiem NavigationBar. Pasek jest widoczny tylko na głównych trasach, natomiast ekrany pomocnicze, takie jak edytor planu czy aktywny trening, są otwierane przez NavHost.



Rysunek 4. Mapa nawigacji aplikacji GymApp.

Ekran	Trasa	Najważniejsze elementy UI
Opcje	settings	Przełączniki motywu, powiadomień, niewygaszania ekranu, wibracji oraz przycisk resetu danych.
Plany	plans	Lista planów, przycisk dodawania, edycja przez kliknięcie karty, usuwanie przez dialog.
Kalendarz	home	Miesięczny kalendarz, oznaczenia dni z treningami, lista planów dla daty, przycisk Start.
Wyniki	stats	Wykres radarowy, wybór ćwiczenia, filtry 14/28 dni, dwa wykresy liniowe.
Profil	profile	Awatar, dane użytkownika, galeria zdjęć progresu, dialog wagi.
Wybór ćwiczeń	exercise_search	Pole wyszukiwania, zakładki grup mięśniowych, lista z checkboxami.
Edytor planu	plan_editor	Tytuł planu, ustawienia ćwiczeń, serie, powtórzenia, ciężar i tempo.
Aktywny trening	active_workout/{planId}	Karty ćwiczeń, pola kg/powtórzeń, checkboxy serii, stoper, minutnik przerwy.

5.2. Interfejsy zewnętrzne

5.2.1. Interfejsy sprzętowe

Aplikacja nie wymaga specjalistycznego sprzętu. Wykorzystuje standardowe urządzenie Android z ekranem dotykowym i lokalną pamięcią. Do wyboru zdjęć używany jest systemowy mechanizm wyboru plików/obrazów.

5.2.2. Interfejsy programistyczne

Interfejs	Opis użycia
Jetpack Compose	Deklaratywne tworzenie ekranów, kart, list, dialogów, wykresów Canvas i dolnej nawigacji.
Navigation Compose	Obsługa tras ekranów i przekazywania planId oraz exerciseIds.
AndroidX Lifecycle ViewModel	Utrzymanie stanu ekranów i uruchamianie operacji w viewModelScope.
Kotlin Coroutines / Flow	Reaktywne strumienie danych, StateFlow i operacje suspend.
Room	Mapowanie encji na tabele SQLite, DAO, relacje i strumienie Flow.
Coil Compose	Wyświetlanie zdjęć zapisanych w prywatnej pamięci aplikacji.

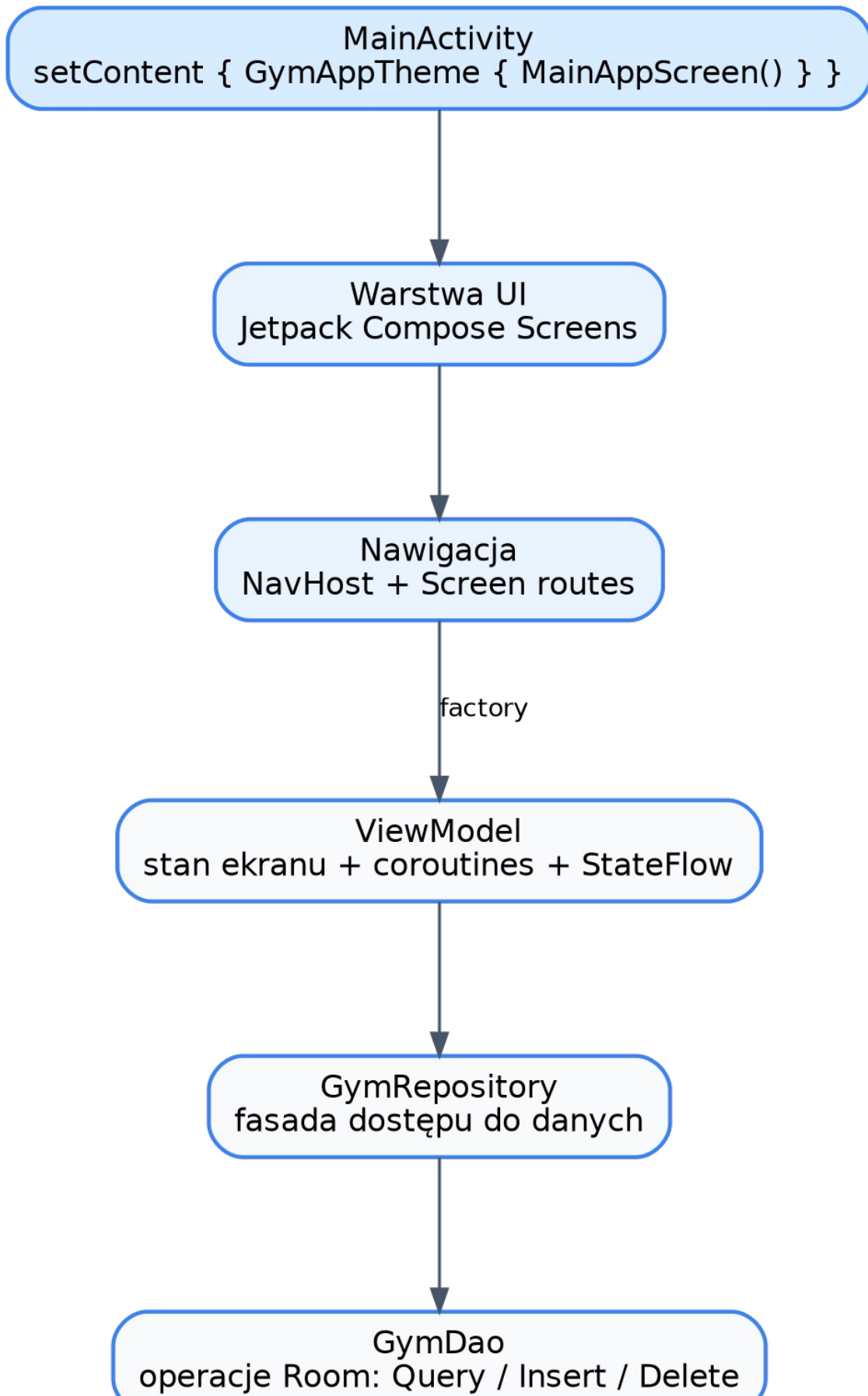
5.2.3. Interfejsy komunikacyjne

System nie korzysta z komunikacji sieciowej. Komunikacja w aplikacji odbywa się lokalnie: ekran komunikuje się z ViewModelem przez StateFlow i funkcje zdarzeń, ViewModel z repozytorium przez metody Kotlin, repozytorium z DAO przez Room, a DAO z SQLite przez wygenerowany kod Room.

6. Wymagania pozafunkcjonalne

ID	Wymaganie	Priorytet	Weryfikacja
WNF-01	Aplikacja powinna uruchamiać się bez konieczności logowania i połączenia z internetem.	W	Tryb offline, uruchomienie na emulatorze.
WNF-02	Interfejs powinien być czytelny na typowym ekranie smartfona, z wyraźnymi kartami i przyciskami.	W	Przegląd ekranów i test ręczny.
WNF-03	Operacje bazodanowe nie powinny blokować głównego wątku UI.	W	Kontrola użycia suspend, Flow i viewModelScope.
WNF-04	Dane użytkownika powinny pozostawać lokalnie na urządzeniu.	W	Brak konfiguracji sieci i backendu.
WNF-05	Aplikacja powinna obsługiwać jasny i ciemny motyw.	P	Przełączenie opcji Ciemny Motyw.
WNF-06	Kod powinien być podzielony na warstwy UI, ViewModel, repository, DAO i entity.	W	Inspekcja struktury pakietów.
WNF-07	Model danych powinien mieć jawne klucze główne i relacje kaskadowe tam, gdzie dane są zależne.	W	Inspekcja encji Room.
WNF-08	Wykresy w aplikacji i dokumentacji powinny być czytelne i nie mogą mieć nachodzących etykiet.	W	Render dokumentu i kontrola ręczna.
WNF-09	Projekt powinien dać się zbudować w Android Studio bez dodatkowego backendu.	W	Import projektu i Gradle sync.
WNF-10	Przyszłe migracje bazy powinny zachowywać dane użytkownika.	R	Dodanie migracji Room zamiast destrukcyjnej migracji.

7. Architektura techniczna



7.1. Struktura pakietów

Pakiet / katalog	Rola
<code>com.example.gymapp</code>	Punkt wejścia aplikacji: MainActivity.
<code>data.local.dao</code>	Interfejs GymDao z zapytaniami i operacjami Room.
<code>data.local.entity</code>	Encje Room odwzorowujące tabele SQLite.
<code>data.local</code>	GymDatabase i InitialExerciseData.
<code>data.repository</code>	GymRepository jako fasada nad DAO.
<code>domain.model</code>	Proste modele domenowe Exercise, PlanExercise, WorkoutPlan i WorkoutSession.
<code>ui.main</code>	MainAppScreen z dolną nawigacją.
<code>ui.navigation</code>	Screen i AppNavHost, czyli definicja tras i tworzenie ViewModeli.
<code>ui.screens.*</code>	Ekrany funkcjonalne oraz ViewModele dla poszczególnych modułów.
<code>ui.theme</code>	Motyw, kolory, typografia i globalny stan motywu.
<code>utils</code>	GymViewModelFactory do wstrzykiwania repository do ViewModeli.

7.2. Punkt wejścia aplikacji

MainActivity dziedziczy po ComponentActivity. W metodzie onCreate wywołuje setContent, nakłada GymAppTheme i uruchamia MainAppScreen. Oznacza to, że cała aplikacja jest tworzona w Compose, bez klasycznych layoutów XML dla ekranów.

7.3. Nawigacja i tworzenie ViewModeli

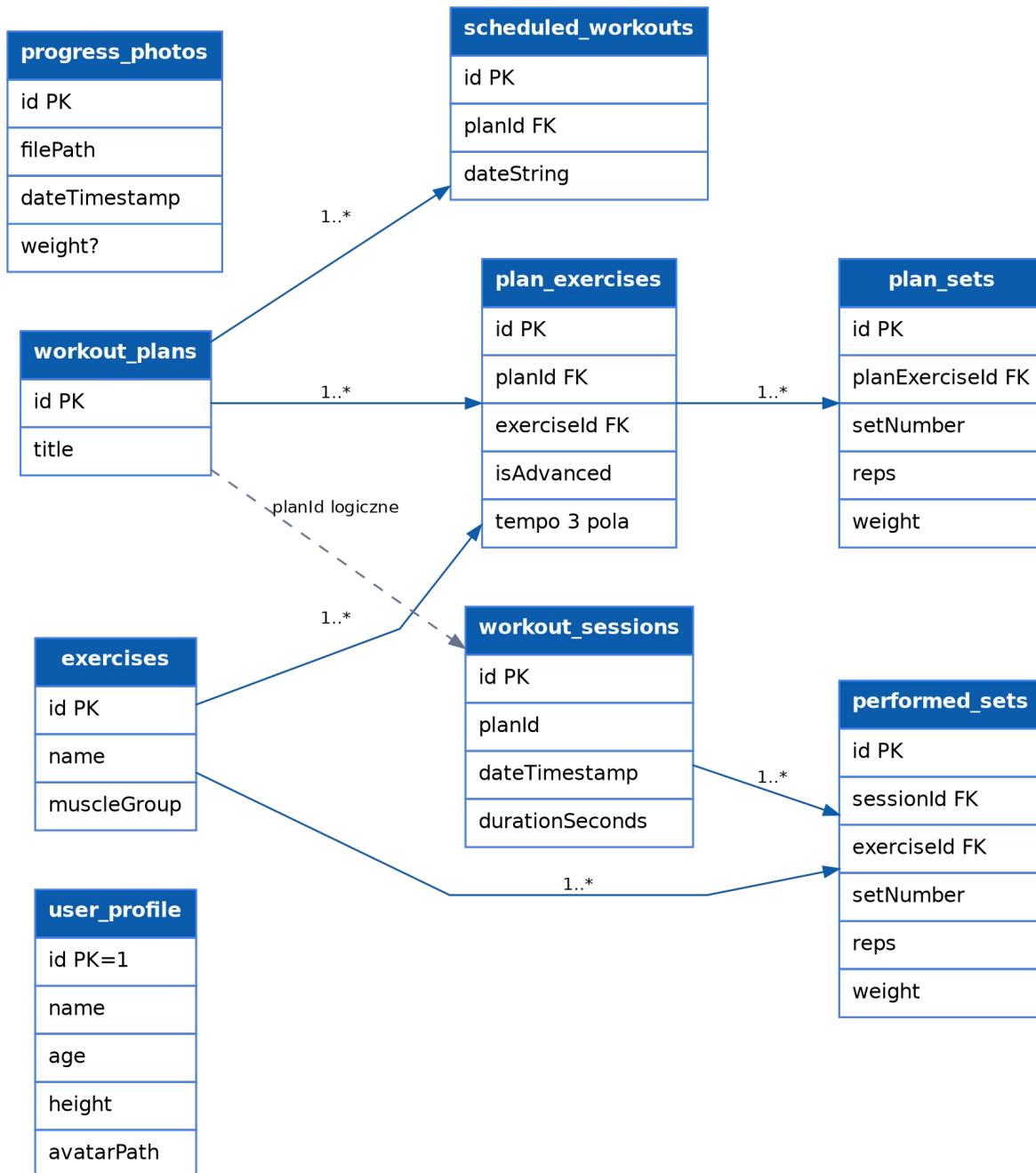
AppNavHost tworzy instancję bazy danych przez GymDatabase.getDatabase(context), następnie tworzy GymRepository oraz GymViewModelFactory. Dla każdej trasy pobierany jest odpowiedni ViewModel przez viewModel(factory = factory). Dzięki temu ekrany nie tworzą DAO bezpośrednio, tylko korzystają z warstwy pośredniej.

7.4. Warstwa danych

GymRepository udostępnia metody odpowiadające przypadkom użycia: pobranie ćwiczeń, zapis planu, zapis serii, pobranie historii, obsługę kalendarza, zdjęć i profilu. Repository nie zawiera złożonej logiki biznesowej; jego główną rolą jest schowanie szczegółów DAO przed ViewModelami.

8. Model danych

Model danych jest oparty na Room. Najważniejsze zależności są zdefiniowane przez klucze obce z kaskadowym usuwaniem: plan usuwa ćwiczenia planu, ćwiczenie planu usuwa serie planowane, sesja usuwa wykonane serie, a plan usuwa wpisy kalendarza.



Rysunek 6. Model danych aplikacji GymApp.

Ważna uwaga do modelu danych

Pole planId w tabeli workout_sessions jest logicznym odniesieniem do planu, ale w obecnym kodzie encji nie ma jawnej adnotacji ForeignKey dla WorkoutSessionEntity. W dokumentacji pokazano tę relację linią przerywaną.

8.1. Słownik tabel

Tabela	Znaczenie	Najważniejsze pola	Uwagi
exercises	Słownik ćwiczeń	id, name, muscleGroup	Wypełniana przy pierwszym utworzeniu bazy.

workout_plans	Plany treningowe	id, title	Główna tabela planów użytkownika.
plan_exercises	Ćwiczenia w planie	id, planId, exerciseId, isAdvanced, tempo	Łączy plan z ćwiczeniami i parametrami tempa.
plan_sets	Serie planowane	id, planExerciseId, setNumber, reps, weight	Szczegółowe serie dla ćwiczenia w planie.
scheduled_workouts	Kalendarz treningów	id, planId, dateString	Przypisanie planu do daty ISO.
workout_sessions	Sesje treningowe	id, planId, dateTimeStamp, durationSeconds	Jedno wykonanie planu.
performed_sets	Serie wykonane	id, sessionId, exerciseId, setNumber, reps, weight	Rzeczywiste wyniki użytkownika.
progress_photos	Zdjęcia progresu	id, filePath, dateTimeStamp, weight	Metadane zdjęć zapisanych w filesDir.
user_profile	Profil	id, name, age, height, avatarPath	Jeden rekord profilu użytkownika.

8.2. Operacje DAO

Obszar	Przykładowe operacje
Ćwiczenia	insertExercises, getAllExercises, searchExercises.
Plany	insertWorkoutPlan, updateWorkoutPlan, deleteWorkoutPlan, getAllWorkoutPlans, getWorkoutPlanById.
Ćwiczenia planu	insertPlanExercise, deletePlanExercisesByPlanId, getPlanExercisesByPlanId.
Serie planowane	insertPlanSets, getPlanSetsByExerciseIds.
Sesje i historia	insertWorkoutSession, insertPerformedSets, getAllSessions, getAllPerformedSets, getExerciseHistoryForStats.
Kalendarz	insertScheduledWorkout, getAllScheduledWorkouts, deleteScheduledWorkout.
Zdjęcia i profil	insertProgressPhoto, getAllProgressPhotos, getUserProfile, upsertUserProfile.
Reset	clearAllWorkoutPlans, clearAllSessions, clearAllScheduled, clearAllPhotos, clearUserProfile.

9. Testowanie i jakość

Projekt zawiera standardowe zależności testowe Android/JUnit/Compose UI Test w konfiguracji Gradle. Dla pełnego odbioru projektu zaleca się wykonanie testów ręcznych oraz dodanie testów jednostkowych dla logiki ViewModeli i testów instrumentacyjnych dla najważniejszych scenariuszy UI.

9.1. Plan testów ręcznych

ID	Scenariusz testowy	Oczekiwany wynik
T-01	Pierwsze uruchomienie aplikacji.	Baza zostaje utworzona, słownik ćwiczeń jest dostępny.
T-02	Dodanie planu z trzema ćwiczeniami.	Plan pojawia się na liście i można go edytować.
T-03	Edycja planu i zmiana liczby serii.	Po zapisie plan zawiera nowe parametry.
T-04	Przypisanie planu do dzisiejszej daty.	Kalendarz pokazuje kropkę i kartę planu.
T-05	Start treningu, zaznaczenie części serii i zakończenie.	Zapisuje się sesja oraz tylko wykonane serie.
T-06	Dodanie zdjęcia progresu z wagą.	Zdjęcie pojawia się w galerii z etykietą kg.
T-07	Reset danych.	Znikają plany, historia, zdjęcia i profil, a katalog ćwiczeń pozostaje.
T-08	Przełączenie ciemnego motywu.	Kolorystyka aplikacji zmienia się zgodnie z ThemeState.

9.2. Propozycje testów automatycznych

- Test ViewModelu ExerciseSearchViewModel: filtr po nazwie, filtr po grupie i toggle zaznaczenia ćwiczenia.
- Test PlanEditorViewModel: konwersja pól tekstowych na wartości domyślne oraz zapis planu z seriami.

- Test `ActiveWorkoutViewModel`: zapis wyłącznie serii oznaczonych jako wykonane.
- Test DAO na bazie in-memory Room: relacje kaskadowe po usunięciu planu i sesji.
- Test UI Compose: przejście Plany -> Wybór ćwiczeń -> Edytor planu -> Lista planów.

10. Wdrożenie i uruchomienie

10.1. Wymagania środowiskowe

Element	Wymaganie / stan projektu
Android Studio	Środowisko zalecane do importu i uruchomienia projektu.
Gradle	Projekt używa Gradle Kotlin DSL oraz wrapperów <code>gradlew/gradlew.bat</code> .
JDK	Konfiguracja kompilacji ustawia zgodność Java 11.
Android SDK	<code>compileSdk 36</code> , <code>targetSdk 36</code> , <code>minSdk 24</code> .
Urządzenie	Emulator lub telefon z Androidem zgodnym z <code>minSdk</code> .

10.2. Kroki uruchomienia

1. Pobrać repozytorium GymApp lub otworzyć je bezpośrednio z systemu kontroli wersji w Android Studio.
2. Poczekać na synchronizację Gradle i pobranie zależności AndroidX, Compose, Room, Coil oraz Navigation Compose.
3. Wybrać emulator lub fizyczne urządzenie Android.
4. Uruchomić konfigurację app.
5. Po pierwszym starcie sprawdzić, czy baza ćwiczeń została zainicjalizowana i czy zakładki aplikacji działają.

10.3. Uwagi do wersji release

W konfiguracji release `minifyEnabled` jest ustawione na false, więc aplikacja nie wykonuje minifikacji kodu. Dla wersji produkcyjnej warto rozważyć włączenie minifikacji, dopracowanie reguł ProGuard, dodanie migracji Room i trwałego zapisu ustawień.

11. Lista spraw otwartych i rekomendacje

ID	Sprawa otwarta / rekomendacja	Priorytet
SO-01	Dodać bezpieczne migracje Room zamiast <code>fallbackToDestructiveMigration</code> .	Wysoki
SO-02	Utrwalać ustawienia powiadomień, wibracji i niewygaszania ekranu np. przez <code>DataStore</code> .	Średni
SO-03	Zintegrować powiadomienia treningowe z <code>WorkManager/AlarmManager</code> .	Średni
SO-04	Poprawić literówkę w nazwie pliku <code>ActiveWrokoutViewModel.kt</code> i <code>ScheduledWourkoutEntity.kt</code> .	Niski
SO-05	Urealnić daty w statystykach postępu przez pełne połączenie <code>performed_sets</code> z <code>workout_sessions</code> .	Wysoki
SO-06	Dodać eksport/import danych użytkownika.	Średni
SO-07	Dodać walidację pól liczbowych w UI przed zapisem.	Średni
SO-08	Dodać testy jednostkowe <code>ViewModeli</code> i testy DAO na bazie in-memory.	Wysoki
SO-09	Uspójnić język interfejsu: część etykiet jest po polsku, część po angielsku.	Średni
SO-10	Rozważyć relację <code>ForeignKey</code> dla <code>WorkoutSessionEntity.planId</code> .	Średni

Dodatek A. Słownik pojęć i terminów

Pojęcie	Definicja
Android Activity	Podstawowy komponent Androida reprezentujący ekran lub punkt wejścia aplikacji.
Jetpack Compose	Deklaratywny framework UI dla Androida, w którym ekran opisuje się funkcjami @Composable.
ViewModel	Klasa przechowująca stan ekranu i logikę reakcji na zdarzenia UI.
StateFlow	Reaktywny strumień stanu z Kotlin Coroutines, obserwowany przez UI.
Room	Biblioteka Android do pracy z SQLite przez encje, DAO i bazę danych.
DAO	Data Access Object - interfejs metod odczytu i zapisu danych.
Repository	Warstwa pośrednia oddzielająca logikę ViewModeli od szczegółów bazy danych.
1RM	Szacowany ciężar maksymalny na jedno powtórzenie. W projekcie używany jest wzór Brzyckiego.
Tempo ćwiczenia	Trzy wartości opisujące czas fazy ekscentrycznej, izometrycznej i koncentrycznej.
Kaskadowe usuwanie	Mechanizm relacji bazy danych, w którym usunięcie rekordu nadrzędnego usuwa rekordy zależne.

Dodatek B. Słownik danych

ExerciseEntity / exercises

Pole	Typ	Znaczenie
id	Int	Klucz główny, generowany automatycznie
name	String	Nazwa ćwiczenia
muscleGroup	String	Grupa mięśniowa

WorkoutPlanEntity / workout_plans

Pole	Typ	Znaczenie
id	Int	Klucz główny, generowany automatycznie
title	String	Tytuł planu

PlanExerciseEntity / plan_exercises

Pole	Typ	Znaczenie
id	Int	Klucz główny
planId	Int	Klucz obcy do workout_plans
exerciseId	Int	Klucz obcy do exercises
isAdvanced	Boolean	Tryb zaawansowany
eccentricTempo	Int	Tempo fazy ekscentrycznej
isometricTempo	Int	Tempo fazy izometrycznej
concentricTempo	Int	Tempo fazy koncentrycznej

PlanSetEntity / plan_sets

Pole	Typ	Znaczenie
id	Int	Klucz główny
planExerciseId	Int	Klucz obcy do plan_exercises
setNumber	Int	Numer serii
reps	Int	Planowane powtórzenia
weight	Double	Planowany ciężar

ScheduledWorkoutEntity / scheduled_workouts

Pole	Typ	Znaczenie
id	Int	Klucz główny
planId	Int	Klucz obcy do workout_plans
dateString	String	Data ISO YYYY-MM-DD

WorkoutSessionEntity / workout_sessions

Pole	Typ	Znaczenie
id	Int	Klucz główny
planId	Int	Identyfikator planu
dateTimeStamp	Long	Data sesji w milisekundach
durationSeconds	Int	Czas trwania w sekundach

PerformedSetEntity / performed_sets

Pole	Typ	Znaczenie
id	Int	Klucz główny
sessionId	Int	Klucz obcy do workout_sessions
exerciseId	Int	Klucz obcy do exercises
setNumber	Int	Numer serii
reps	Int	Wykonane powtórzenia
weight	Double	Rzeczywisty ciężar

ProgressPhotoEntity / progress_photos

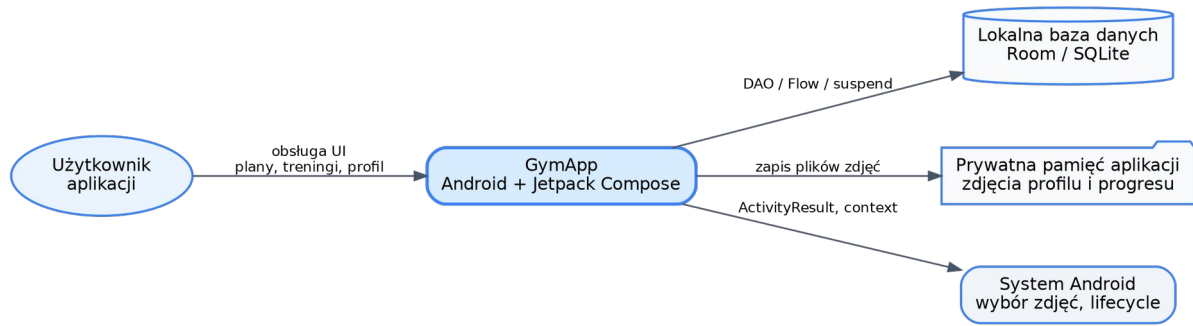
Pole	Typ	Znaczenie
id	Int	Klucz główny
filePath	String	Ścieżka do zdjęcia w pamięci aplikacji
dateTimeStamp	Long	Data dodania
weight	Double?	Opcjonalna waga

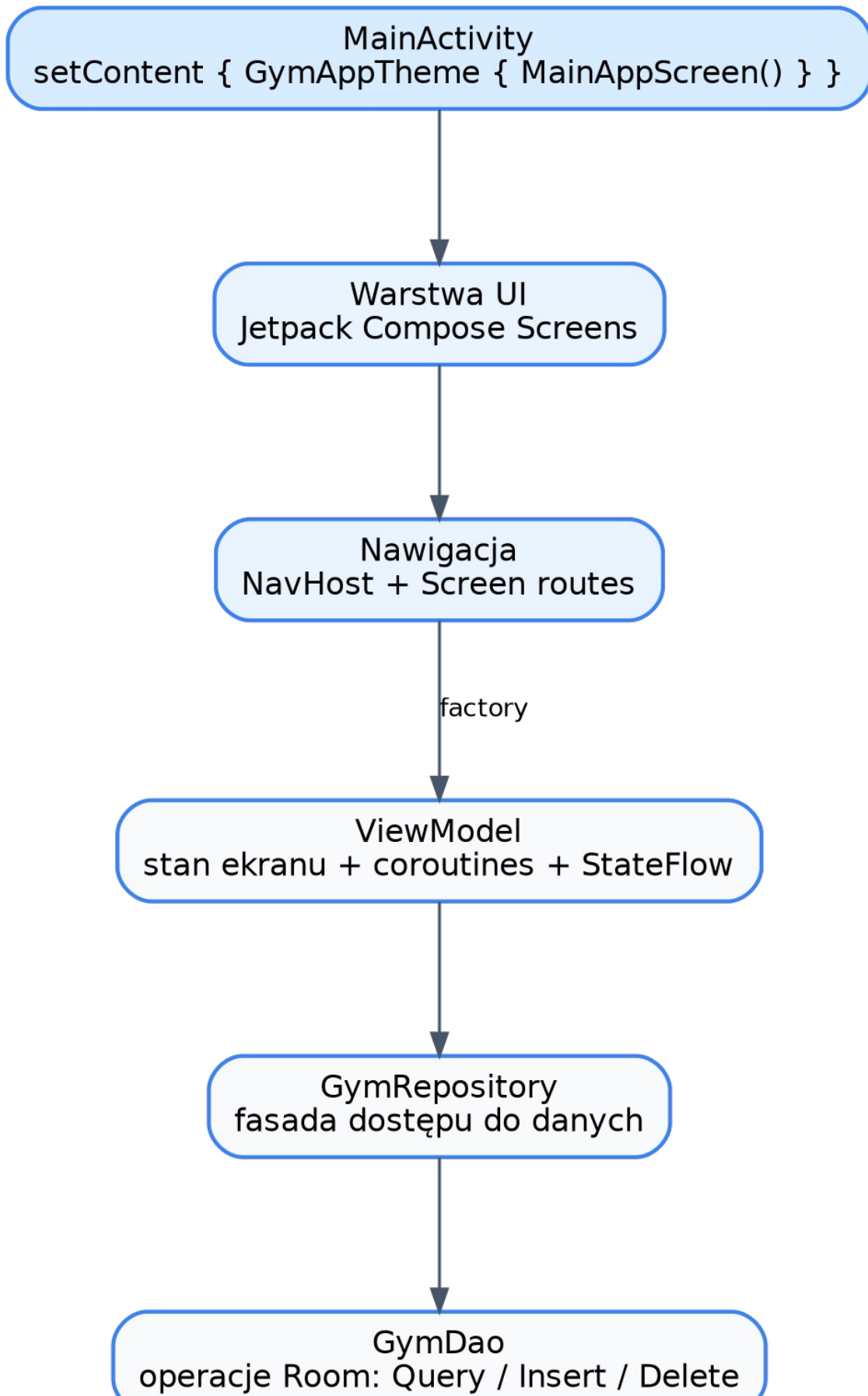
UserEntity / user_profile

Pole	Typ	Znaczenie
id	Int	Stały identyfikator profilu, domyślnie 1
name	String	Imię lub nick
age	Int	Wiek
height	Int	Wzrost w cm
avatarPath	String?	Opcjonalna ścieżka awatara

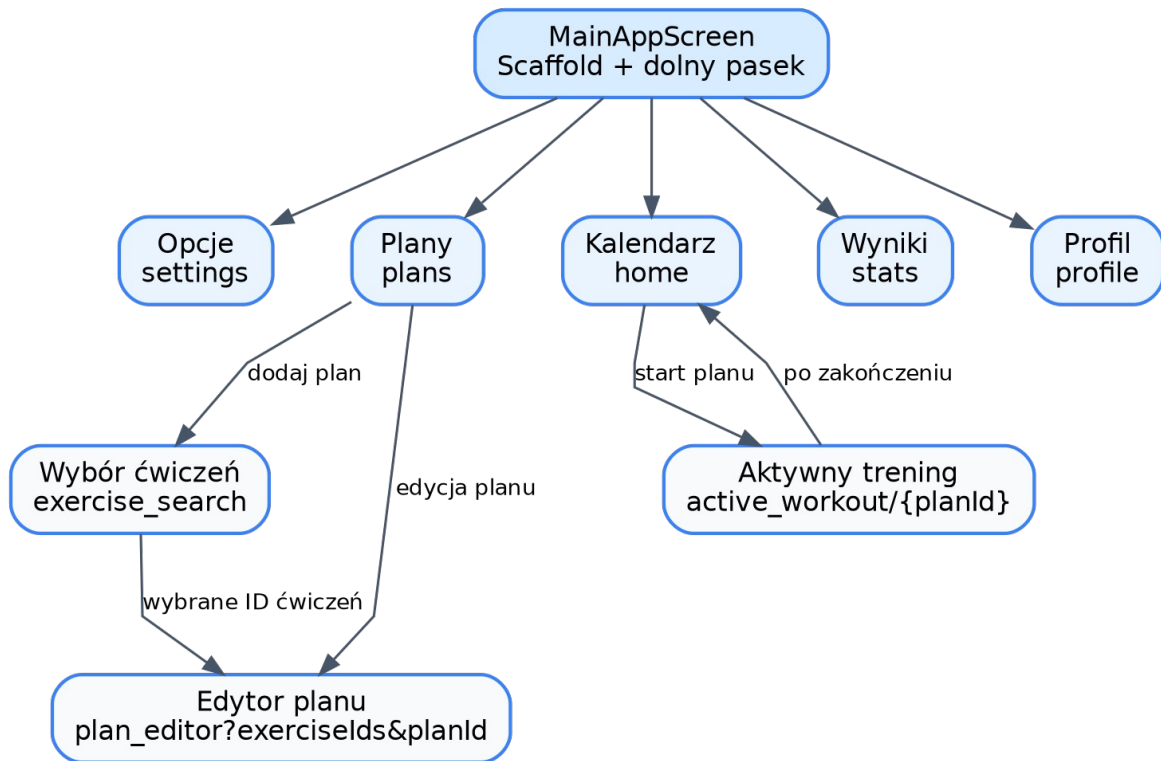
Dodatek C. Modele analizy

Poniżej zebrano najważniejsze diagramy analityczne użyte w dokumentacji. Każdy diagram został przygotowany osobno, z większym odstępem między elementami, aby zachować czytelność i uniknąć nachodzenia opisów.

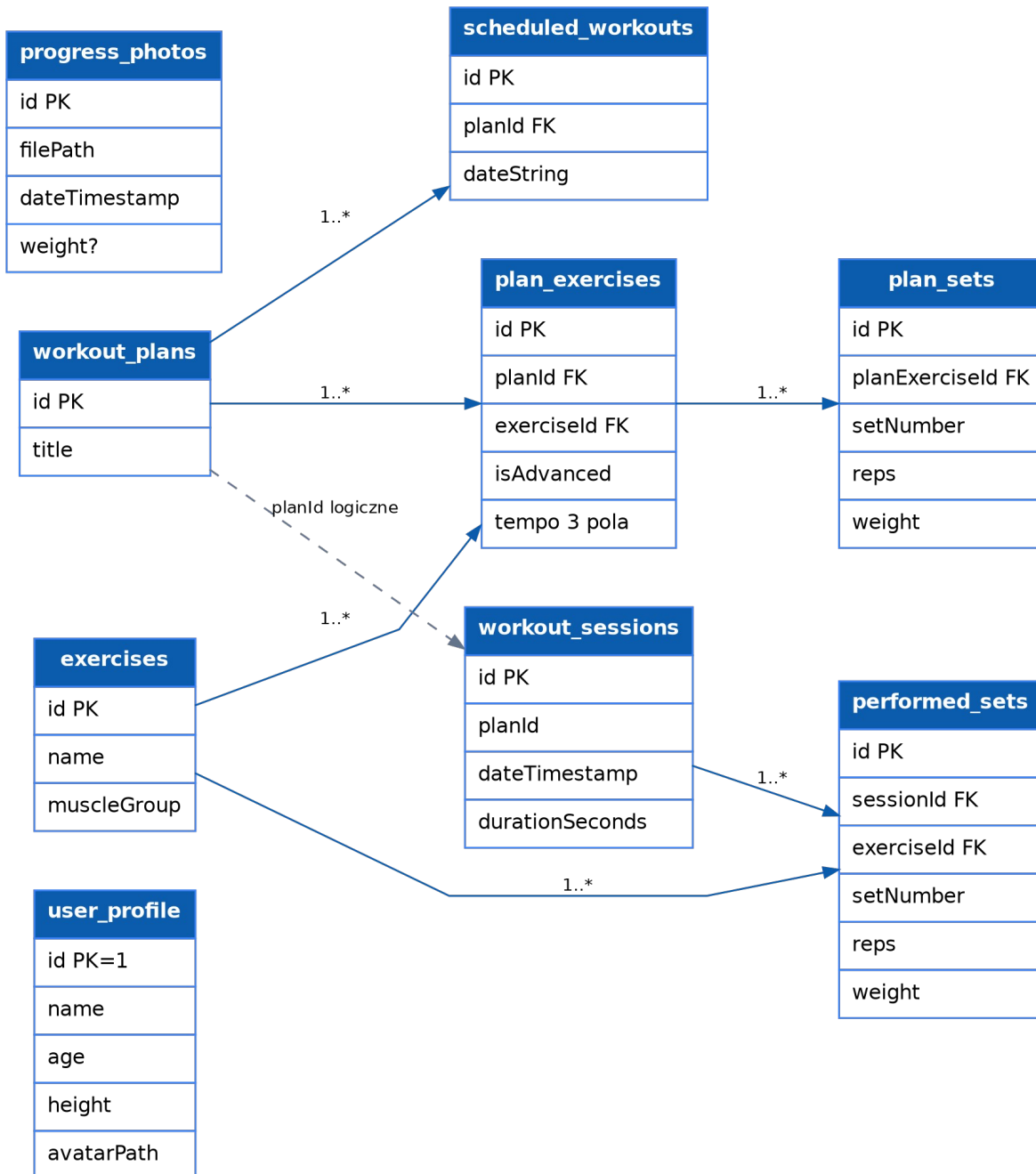
*Rysunek C.1. Kontekst systemu.*



Rysunek C.2. Architektura warstwowa.



Rysunek C.3. Nawigacja.



Rysunek C.4. Model danych.

Podsumowanie

GymApp jest spójną aplikacją mobilną typu lokalny dziennik treningowy. Najmocniejsze elementy projektu to prosty podział warstw, użycie Compose i Room, jasny model planów oraz aktywnego treningu, a także rozbudowanie aplikacji o profil, zdjęcia progresu i statystyki. Najważniejsze techniczne kierunki dalszego rozwoju to bezpieczne migracje bazy danych, trwały zapis ustawień, dopracowanie statystyk datowanych rzeczywistymi sesjami oraz pokrycie logiki testami automatycznymi.